

ShelfAware — Local Supermarket Management System

1. Introduction

Most small supermarkets still track products, stock, suppliers, and sales on paper. This works until it doesn't — records are slow to search, mistakes creep in, and the owner has no real picture of what is selling or what is about to run out. This project replaces that with a single digital system: a C# / .NET application backed by a SQL Server database.

The system handles the daily operations of a small shop:

- **Product management** — add, update, delete and list products (ID, name, brand, barcode, price, quantity, category, supplier, expiry and restock dates).
- **Category and supplier management** — full create, read, update and delete.
- **Stock control** — restocking, adjustments and automatic low-stock alerts. Every change is written to a stock-movement log.
- **Sales** — multi-item sales that reduce stock automatically, print a receipt and save the sale to history.
- **Searching** — four different search strategies over the product catalogue.
- **Reporting** — low-stock items, sales by product, products by category, and supplier stock.

I built the application as an **ASP.NET Core (Razor Pages)** web app, which is one of the permitted UI options. The business logic sits in a separate `Supermarket.Core` class library, so the data structures, algorithms and services can be reused and unit-tested without touching the UI. Persistence is handled by **Entity Framework Core** on top of **SQL Server**.

2. Justification of Data Structures and Algorithms

The brief asks for custom data structures instead of built-in collections for the main data structure. Products are loaded from the database into custom structures, and all searching runs against those. I implemented three structures, each picked to make one specific operation fast.

The brief asks for custom data structures instead of built-in collections for the main data structure. Products are loaded from the database into custom structures, and all searching runs against those. I implemented three structures, each picked to make one specific operation fast.

2.1 Custom singly linked list (`CstmLinkedList<T>`)

This is the main in-memory store for products. I went with a linked list because the product set changes often — products get added and removed — and a linked list appends in **$O(1)$** without the resizing cost you pay with a dynamic array. It implements `IEnumerable<T>` so the rest of the program can iterate over it, and it exposes `ToArray()` so sorting and binary search can index the data. This replaces `List<T>` and satisfies the "no built-in collection for the main structure" requirement.

2.2 Binary search tree (`BST<TKey, TValue>`)

Used to search products **by name**. A BST keeps keys in order, which gives **$O(\log n)$** average search time, and an in-order traversal gives me the products alphabetically for free. Duplicate names are stored in a small list on the matching node, so one search returns every product with that name.

2.3 Hash table (`Hashtable<TKey, TValue>`)

Used for **barcode lookup**, which is the most time-critical operation at the till. I used separate chaining — each bucket is a `CstmLinkedList` — which gives **$O(1)$** average lookup. The table resizes once the load factor goes past 0.75, keeping the chains short.

2.4 Algorithms

Algorithm	Where it is used	Complexity
-----------	------------------	------------

Linear search	"contains" name search, category, supplier	$O(n)$
Binary search	exact name search on a sorted array	$O(\log n)$
QuickSort	sorting products by name before binary search	$O(n \log n)$ average
BST search	exact name search	$O(\log n)$ average
Hash lookup	barcode search	$O(1)$ average

Having both linear and binary search covers the "at least two search algorithms" requirement, and it lets me contrast a simple $O(n)$ scan against a faster $O(\log n)$ approach that needs a sort first.

3. Time Complexity Analysis with Pseudo-code

Add product — append to the linked list, then persist.

```
add(product):  
    validate(product)           # O(1)  
    list.append(product)        # O(1)  
    db.save(product)           # DB write
```

Time: **O(1)** in memory, plus the database write.

Search by barcode (hash table)

```
get(barcode):  
    i = hash(barcode) mod capacity    # O(1)  
    for entry in buckets[i]:         # short chain, O(1)  
        average  
        if entry.key == barcode: return entry.value  
    return null
```

Time: **O(1)** average. Worst case is **O(n)** if every key lands in the same bucket.

Search by name (BST)

```
search(name):  
    node = root  
    while node != null:  
        if name == node.key: return node.values  
        node = name < node.key ? node.left : node.right  
    return empty
```

Time: **O(log n)** average, **O(n)** worst case if the tree becomes unbalanced.

Search by name (sorted array)

```
sortedSearch(name):  
    arr = list.toArray()           # O(n)  
    quicksort(arr by name)         # O(n log n) average  
    return binarySearch(arr, name) # O(log n)
```

The sort dominates at **$O(n \log n)$** . The lookup itself is only **$O(\log n)$** .

Stock update

```
adjustStock(id, change):  
    p = db.find(id)                #  $O(1)$  by primary  
key  
    if p.quantity + change < 0: error  
    p.quantity += change  
    log movement; db.save()
```

Time: **$O(1)$** in memory, plus the database write.

Record sale

```
record(lines):  
    for (id, qty) in lines:        #  $O(k)$  lines  
        p = db.find(id)  
        if p.quantity < qty: error # validate  
everything first  
    p.quantity -= qty; add sale item; log movement  
    total = sum(line totals); db.save()
```

Time: **$O(k)$** for k items in the basket.

Low-stock report

```
lowStock(threshold):  
    return [p for p in products if p.quantity <= threshold]
```

Time: **$O(n)$** .

4. Database Design

The SQL Server database has six tables, created and managed through Entity Framework Core:

- **Categories** (Id, Name)
- **Suppliers** (Id, Name, Contact)
- **Products** (Id, Name, Brand, Barcode — unique, Price, Quantity, ExpiryDate, RestockDate, CategoryId → Categories, SupplierId → Suppliers)
- **StockMovements** (Id, ProductId → Products, Change, Reason, Date)
- **Sales** (Id, Date, Total)
- **SaleItems** (Id, SaleId → Sales, ProductId → Products, Quantity, UnitPrice)

The relationships are straightforward: a category and a supplier each have many products, a sale has many sale items, and a product has many stock movements. A **unique index on Barcode** enforces the no-duplicate-barcode rule at the database level. Prices are stored as `decimal(10,2)`. EF Core generates the schema, and I have also included it as a standalone script (`Database/CreateDatabase.sql`). On first run the application

creates the database and seeds sample categories, suppliers and products.

Validation happens in the service layer before anything is saved: name, barcode, supplier and category are required, price must be positive, quantity cannot be negative, the expiry date cannot be in the past, and the barcode must be unique — checked both in code and by the unique index.

5. Testing Approach

Tests are written with **xUnit** using the EF Core **InMemory** provider, so they run without a SQL Server instance and every test gets a fresh database. I covered two areas.

Data structures and algorithms (DataStructureTests):

- Linked list add, remove, count and array conversion.
- BST search, duplicate-key handling, and sorted in-order traversal.
- Hash table put, get, remove, overwrite, and resize integrity across 50 keys.
- Linear search collecting all matches; QuickSort followed by binary search for both a hit and a miss.

Supermarket workflows (ServiceTests):

- Adding a product persists it.
- A duplicate barcode is rejected.
- A non-positive price is rejected (theory test with 0 and -2).
- A stock update reduces quantity and blocks going negative.
- Barcode and name searches return the right product, and null or empty when nothing matches.
- Recording a sale reduces stock, computes the total and saves the history.
- A sale that exceeds available stock is rejected and stock is left unchanged.
- The low-stock report flags products at or below the threshold.

These map directly onto the operations named in the brief: adding products, updating stock, searching by barcode and name, recording sales, and generating low-stock reports.

6. Conclusion, Limitations and Reflection

The system meets the coursework requirements. It manages products, categories, suppliers, stock and sales; persists everything to SQL Server through Entity Framework; implements three custom data structures and several search algorithms with documented complexity; validates input; and is covered by unit tests.

Limitations. The BST is not self-balancing, so with a bad insertion order the search degrades to $O(n)$. A balanced tree such as an AVL or red-black tree would guarantee $O(\log n)$. The in-memory search structures are also rebuilt from the database on demand rather than kept in sync incrementally — for a very large catalogue it would make more sense to cache them and update them as the data changes.

Future improvements. Receipt printing, barcode-scanner integration, customer accounts with loyalty, and richer sales analytics (trends, profit margins) would push the system towards a full point-of-sale product, as the brief suggests.

7. References

- Microsoft (2024) *Entity Framework Core documentation*. Available at: <https://learn.microsoft.com/ef/core/> (Accessed: 28 June 2026).
- Microsoft (2024) *.NET documentation*. Available at: <https://learn.microsoft.com/dotnet/> (Accessed: 28 June 2026).
- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) *Introduction to Algorithms*. 3rd edn. Cambridge, MA: MIT Press.